Naive Bayes: The Intuition

Contents

1. Overview 2
2. The method on one picture 2
3. The data 4
4. The strategy: compare posteriors 5
5. Applying Bayes' theorem 5
6. The problem: specific combinations are rare 6
7. The naive assumption 7
8. Computing the scores 9
9. One zero count 10
10. Summary 12
11. Sources 12
12. Key terms 12

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Classification problems; Probability density function (PDF); Normal distribution; Independent and mutually exclusive events; Bayes' theorem.

1. Overview

Key point: Naive Bayes computes, for each class, a score: the class's prior multiplied by one **likelihood** (G-1086) per feature. It predicts the class with the largest score. To make this possible, it assumes the features are independent within each class.

With **conditional probability** (G-444), independence and **Bayes' theorem** (G-269) in place, we can build the **Naive Bayes classifier** (G-1297), over three Notes: this Note develops the intuition, the next Note derives the mathematics, and the one after codes it.

This Note covers:

- the whole method on one picture, with word counts (section 2);
- a small dataset of cricket matches (section 3);
- the strategy: compare the **posteriors** (G-1536) of the classes (section 4);
- Bayes' theorem, and why the **evidence** (G-718) can be dropped (section 5);
- the problem with exact combinations of feature values (section 6);
- the **naive assumption** (G-1296) that solves the problem (section 7);
- the scores for the new match, one factor at a time (section 8);
- what one zero count does to a score (section 9).

2. The method on one picture

Key point: A likelihood P(word | class) is a bar height divided by the class's total. A message's score for a class is the prior times the likelihood of each of its words. The larger score wins.

Suppose we want to separate normal messages from spam. We have 12 normal messages and 6 spam messages. The method has three steps (Figure 1):

1. **Count.** For each class, count how often each word appears. In the 12 normal messages, "hello" appears 9 times out of 20 words. In the 6 spam messages, "hello" appears 2 times out of 10 words.
2. **Turn counts into likelihoods.** Divide each bar by the total number of words of its class (the bar | reads "given": the probability of "hello" given a normal message):

$$P(\text{hello} \mid \text{normal}) = 9/20 = 0.45$$

$$P(\text{hello} \mid \text{spam}) = 2/10 = 0.20$$

Each of these numbers is a **likelihood** (G-1086): how probable a word is if the message belongs to that class.

3. **Score a new message.** For the message "hello free", start from the share of messages in each class, the **prior** (G-1565): $12/18 = 0.67$ for normal and $6/18 = 0.33$ for spam. Multiply the prior by the likelihood of each word in the message.

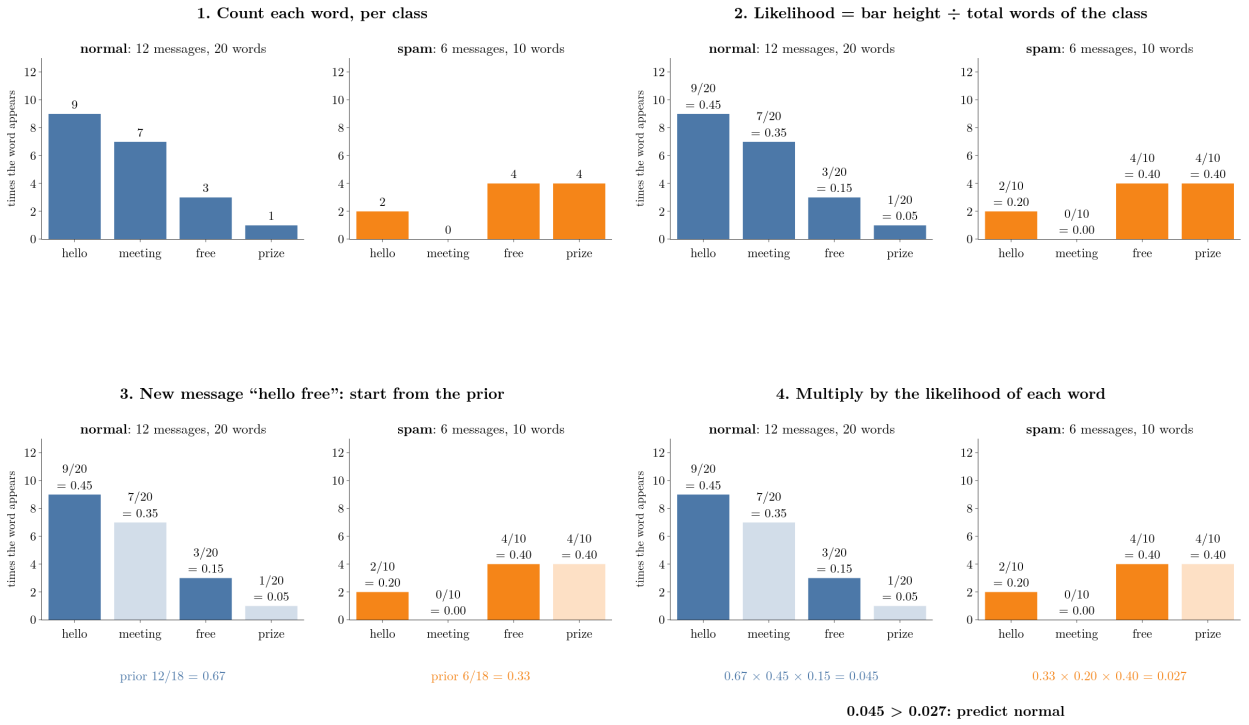


Figure 1: Naive Bayes on word counts. 1: count each word per class. 2: each likelihood is a bar height divided by the class total. 3 and 4: the message "hello free" starts from the prior and is multiplied by the two highlighted bars: 0.045 for normal, 0.027 for spam. Idea after StatQuest, "Naive Bayes, Clearly Explained!!!"; the numbers are ours.

In Figure 1, watch the last frame. Only the bars of the words in the message are used:

$$\text{score}(\text{normal}) = 0.67 \times 0.45 \times 0.15$$

$$= 0.045$$

$$\text{score}(\text{spam}) = 0.33 \times 0.20 \times 0.40$$

$$= 0.027$$

The normal score is larger, so the message is classified as normal. Each number is a **score** (G-1753): it is not the probability of the class, only proportional to it, and that is enough to pick the winner. Sections 4 to 7 explain why this recipe is right, on a second dataset.

3. The data

> **Key point:** Eight cricket matches, three features (toss, venue, outlook) and the result. Predict the result of a new match.

The main example is a small made-up dataset about one cricket team. Each match is one **observation** (one record, a row of the table). Each has three **features** (input variables, one column each) and one **target** (the output we predict), as Figure 2 shows:

match	toss	venue	outlook	result
1	won	Chennai	sunny	win
2	won	Mumbai	sunny	win
3	lost	Chennai	sunny	win
4	won	Mumbai	overcast	win
5	won	Chennai	sunny	win
6	lost	Mumbai	sunny	loss
7	lost	Chennai	rainy	loss
8	won	Mumbai	rainy	loss
new	lost	Mumbai	sunny	**?**

Figure 2: Eight matches and a new one to predict

- **toss:** did the team win or lose the toss?
- **venue:** Chennai or Mumbai.
- **outlook:** the weather, sunny, overcast or rainy.
- **result:** win or loss, the class to predict.

For a new match the toss is **lost**, the venue is **Mumbai** and the weather is **sunny**. Will the team win?

4. The strategy: compare posteriors

> **Key point:** Compute P(win | features) and P(loss | features); predict whichever is larger.

Naive Bayes asks two questions:

$$P(\text{win} \mid \text{lost}, \text{Mumbai}, \text{sunny})$$

and

$$P(\text{loss} \mid \text{lost}, \text{Mumbai}, \text{sunny})$$

Each is a **posterior** (G-1536): the probability of a class after the features are seen. The commas mean "and": the three conditions hold together, so they could also be written with $\cap$, the sign for "and" between events: "lost $\cap$ Mumbai $\cap$ sunny" means all three hold in the same match. With only two classes the two posteriors add up to 1. If the first is, say, 0.7 and the second 0.3, the classifier predicts a win. With more classes (for example win, loss, draw) the classifier computes one probability per class and again takes the largest.

5. Applying Bayes' theorem

> **Key point:** Each posterior equals **likelihood** (G-1086) $\times$ prior / evidence. The evidence is the same for every class, so it can be dropped.

With A = the class and B = the three feature values, **Bayes' theorem** (G-269) gives

$$P(\text{win} \mid B) = \frac{P(B \mid \text{win}) \times P(\text{win})}{P(B)}$$

and the same with "loss" in place of "win".

The denominator, the **evidence** (G-718), is identical in both: it is the probability of seeing these feature values, whichever the class. Here only 1 of the 8 matches (match 6) had all three values, so the evidence is 1/8 for both classes. Dividing two numbers by the same positive amount does not change which is larger. So for choosing the class, only the **numerator** matters, and this numerator is the score of section 2:

$$\text{score}(\text{class}) = P(\text{features} \mid \text{class}) \times P(\text{class})$$

The **priors** (G-1565), also called **class priors** (G-388), come straight from counting: 5 of the 8 matches were wins and 3 were losses.

$$P(\text{win}) = 5/8$$

$$P(\text{loss}) = 3/8$$

Figure 3 puts both steps in one picture. Watch the shared denominator $P(\text{f})$ being struck out in both **posteriors** (G-1536), and the priors coming from a plain count of the eight matches.

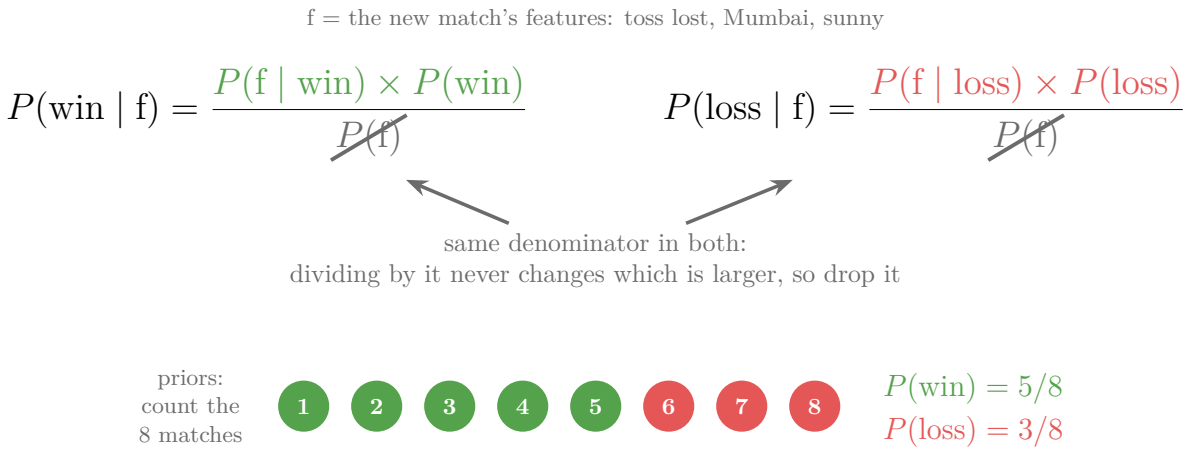


Figure 3: Both posteriors divide by the same evidence P(f), so it can be dropped; the priors are counts, 5 wins and 3 losses out of 8 matches

6. The problem: specific combinations are rare

> **Key point:** No winning match had exactly (lost, Mumbai, sunny), so the **likelihood** (G-1086) P(lost, Mumbai, sunny | win) would be 0.

The likelihood $P(\text{lost}, \text{Mumbai}, \text{sunny} \mid \text{win})$ is a **joint probability** (G-986), the probability of all three conditions together. It asks: among the 5 winning matches, how many had all three conditions at once? None did. So it would be 0, and the score for "win" would be 0 however strongly the other **evidence** (G-718) pointed to a win.

Figure 4 runs this search one condition at a time. Watch the winning matches grey out: one survives "toss lost", none survives "Mumbai", while one losing match (match 6) survives all three.

step 3: also outlook = sunny. Win: 0 of 5 agree

win: 0 of 5 left

match	toss	venue	outlook
1	won	Chennai	sunny
2	won	Mumbai	sunny
3	lost	Chennai	sunny
4	won	Mumbai	overcast
5	won	Chennai	sunny

loss: 1 of 3 left

match	toss	venue	outlook
6	lost	Mumbai	sunny
7	lost	Chennai	rainy
8	won	Mumbai	rainy

Figure 4: Keeping only past matches that agree with the new one on toss, then venue, then outlook. Win: 5, then 1, then 0 of 5 left. Loss: 3, then 2, then 1 of 3 left

The rarity of exact matches is a general problem. A past observation that matches a new one on **every** feature at once is rare. With 3 features the search already failed here; with 10 or 20 features, almost every combination would never have been seen. The estimates would be 0 or based on one or two observations, and the classifier would be useless.

7. The naive assumption

> **Key point:** Assume the features are independent within each class. Then the joint **likelihood** (G-1086) becomes a product of one-feature likelihoods, each estimated from many observations.

The way out is to stop asking for all three conditions at once. We count each feature on its own, and multiply the results, exactly as each word was counted on its own in section 2.

Multiplying is allowed only under an assumption, the **naive assumption** (G-1296): **within each class, the features are independent** of each other. The standard name is **conditional independence** (G-443) of the features given the class. From the definition of independent events, the probability of independent events happening together is the product of their probabilities. So

$$P(\text{lost}, \text{Mumbai}, \text{sunny} \mid \text{win})$$

$$\approx P(\text{lost} \mid \text{win})$$

$$\times P(\text{Mumbai} \mid \text{win})$$

$$\times P(\text{sunny} \mid \text{win})$$

Each factor looks at one feature only, so it is estimated from all the observations of that class. Figure 5 shows the difference from Figure 4: no match is thrown away, and each column is counted on its own over all five wins (or all three losses).

Naive: count each feature on its own, over every match of the class

$1/5 \times 2/5 \times 4/5$ ($\times 5/8$ prior)

match	toss = lost?	venue = Mumbai?	outlook = sunny?
1	won	Chennai	sunny
2	won	Mumbai	sunny
3	lost	Chennai	sunny
4	won	Mumbai	overcast
5	won	Chennai	sunny
count	1/5	2/5	4/5

$2/3 \times 2/3 \times 1/3$ ($\times 3/8$ prior)

match	toss = lost?	venue = Mumbai?	outlook = sunny?
6	lost	Mumbai	sunny
7	lost	Chennai	rainy
8	won	Mumbai	rainy
count	2/3	2/3	1/3

Figure 5: The naive way: within each class, count the matches that agree on each feature separately. Win: 1/5, 2/5, 4/5. Loss: 2/3, 2/3, 1/3

The assumption is rarely exactly true, which is why the method is called **naive**, but it makes the estimates workable (ISL §4.4.4).

What the product gives up is any link between the features. For messages, the words are treated as unrelated to each other, so their order does not matter: "hello free" and "free hello" get the same score. A message is handled as a **bag of words** (G-250). Even so, Naive Bayes separates normal messages from spam well in practice, for two reasons (scikit-learn user guide §1.9):

- each factor is a one-feature estimate, so it needs only a small amount of training data, where the exact-combination count of section 6 needed far more;
- choosing the class only needs the right class to get the largest score, so the probabilities can be off while the winner is still right (when the assumption fails).

An everyday picture: to guess whether a new dish will taste good, we cannot wait for the exact same recipe to have been cooked before. Instead we judge each ingredient on its own record, and combine the verdicts. The naive assumption step derives it properly.

8. Computing the scores

> **Key point:** Win: $5/8 \times 1/5 \times 2/5 \times 4/5 = 0.040$. Loss: $3/8 \times 2/3 \times 2/3 \times 1/3 = 0.056$. Predict loss.

Figure 6 repeats the three steps of section 2 on the matches. Each bar counts the matches of one class with one feature value. The three bars of the new match light up one at a time, and each one multiplies the running product, which starts at the **prior** (G-1565).

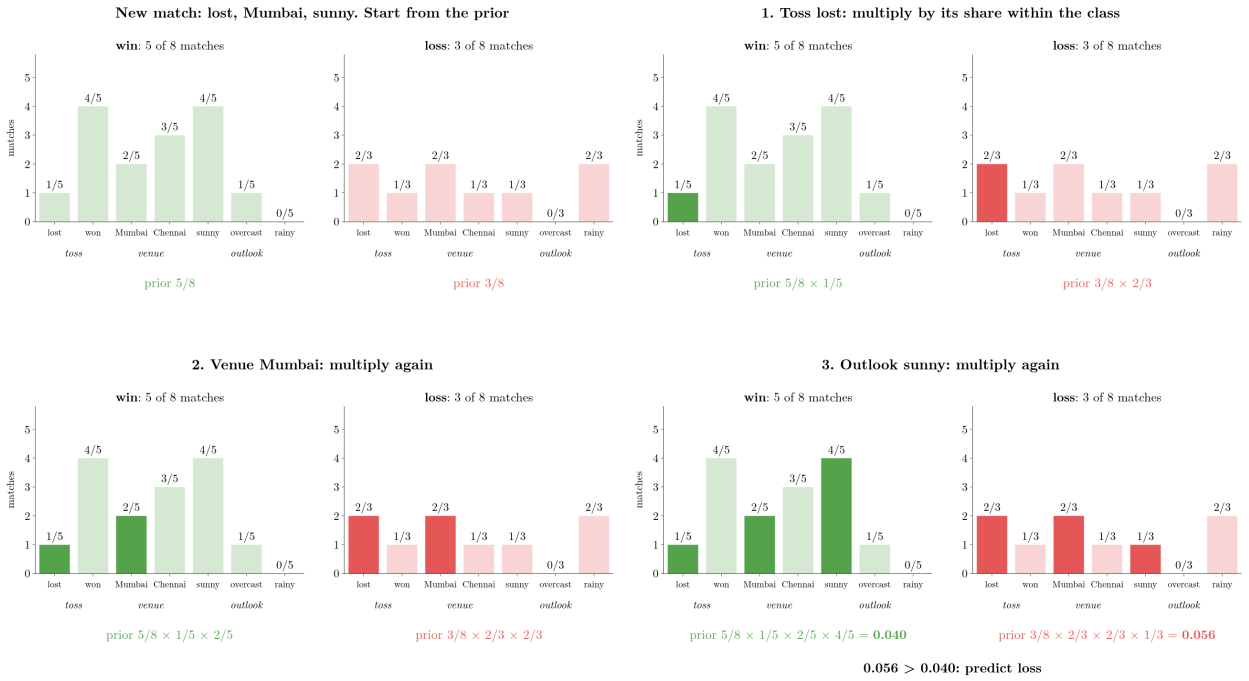


Figure 6: The scores built one factor at a time. Each chart counts one class's matches by feature value. The bars for lost, Mumbai and sunny light up in turn and multiply the prior: win $5/8 \times 1/5 \times 2/5 \times 4/5 = 0.040$, loss $3/8 \times 2/3 \times 2/3 \times 1/3 = 0.056$. Idea after StatQuest, "Naive Bayes, Clearly Explained!!!"; the data are ours.

Win (5 matches): the toss was lost in 1 of them, the venue was Mumbai in 2, and the weather was sunny in 4.

$$\text{score}(\text{win}) = \frac{5}{8} \times \frac{1}{5} \times \frac{2}{5} \times \frac{4}{5}$$

$$= 0.040$$

Loss (3 matches): the toss was lost in 2, Mumbai in 2, sunny in 1.

$$\text{score}(\text{loss}) = \frac{3}{8} \times \frac{2}{3} \times \frac{2}{3} \times \frac{1}{3}$$

$$= 0.056$$

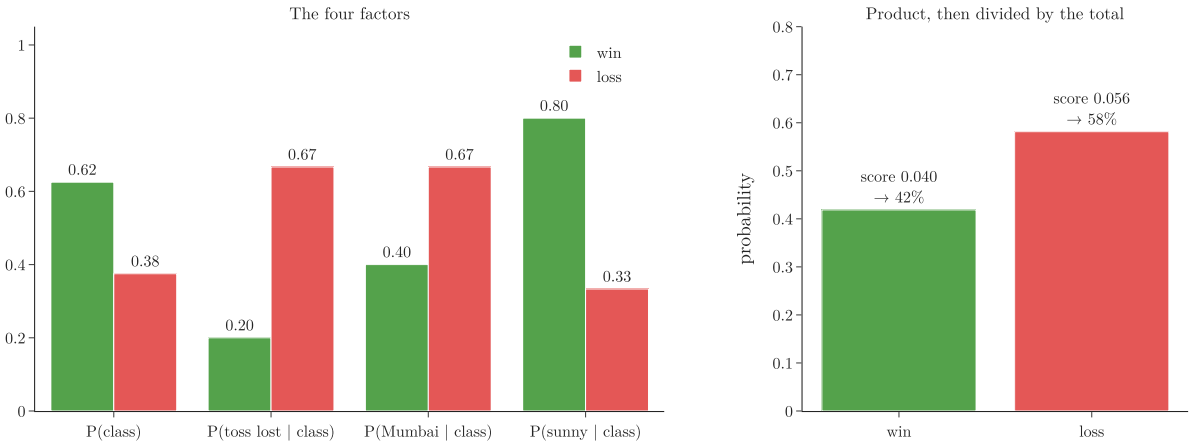


Figure 7: The four factors for each class, and the resulting probabilities

The loss score is larger, so Naive Bayes predicts a **loss**. Dividing each score by their sum turns them into probabilities. For a win:

$$0.040/0.096 = 0.42$$

So 42% win and 58% loss (Figure 7, right).

Figure 7 (left) shows why: winning teams mostly won the toss, played in Chennai and had sunny weather, so "lost the toss" (0.20 against 0.67) and "Mumbai" (0.40 against 0.67) both point to a loss. Sunny weather points to a win (0.80 against 0.33), but not strongly enough to outweigh them.

9. One zero count

> **Key point:** A single **likelihood** (G-1086) of 0 forces the whole score to 0. Adding 1 to every count removes the zeros.

The product has one weak spot. In the word counts of section 2, "meeting" never appears in spam, so:

$$P(\text{meeting} \mid \text{spam}) = 0/10 = 0$$

Take the message "meeting prize prize prize". "Prize" is a typical spam word, yet the spam score is:

$$0.33 \times 0 \times 0.40^3 = 0$$

One zero wipes out everything else, and the message is classified as normal (Figure 8, frame 1).

The fix is to add 1 to every count before dividing (the black boxes in Figure 8). Now the likelihood is no longer 0:

$$P(\text{meeting} \mid \text{spam}) = 1/14$$

The spam score is 0.00108 against 0.00013 for normal, and the message is classified as spam. The **priors** (G-1565) do not change, because the number of messages in each class is the same as before.

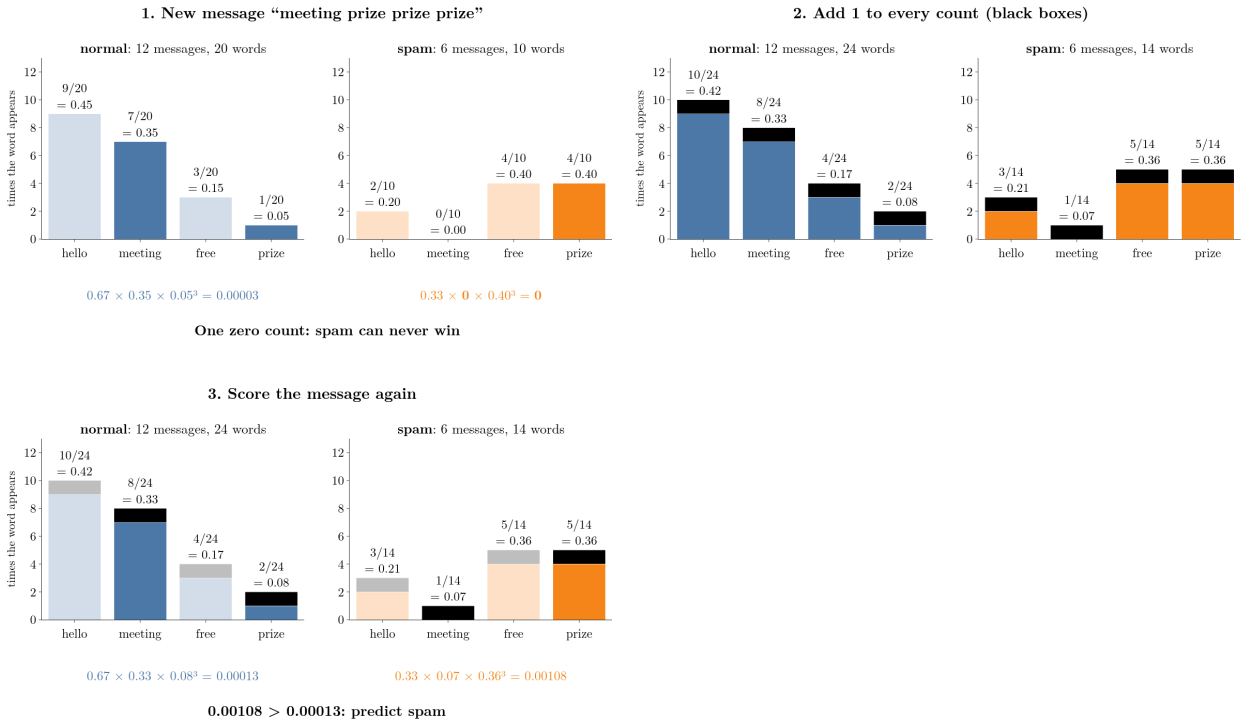


Figure 8: One zero count. 1: "meeting" never appeared in spam, so the spam score of "meeting prize prize prize" is 0. 2: one count is added to every word of both classes. 3: no likelihood is 0 any more, and spam wins, 0.00108 against 0.00013. Idea after StatQuest, "Naive Bayes, Clearly Explained!!!"; the numbers are ours.

Adding a count to every value is **Laplace smoothing** (G-1045). The zero-frequency problem and the fix teach it in full.

10. Summary

Step	Win	Loss
Prior P(class)	5/8	3/8
P(lost | class)	1/5	2/3
P(Mumbai | class)	2/5	2/3
P(sunny | class)	4/5	1/3
Score (product)	0.040	**0.056**

- Naive Bayes picks the class with the largest P(class | features), so for the new match it predicts a loss (score 0.056 against 0.040 for a win).
- A **likelihood** (G-1086) is a count within a class divided by the class total: a bar height, so training is just counting.
- The evidence is shared by all classes, so only likelihood × **prior** (G-1565) is compared, because dividing every score by the same number does not change which is largest.
- Exact combinations of feature values are rare; the **naive assumption** (G-1296) (conditional independence) splits them into one-feature probabilities, so each factor is estimated from all the observations of its class instead of from the one or two that match on every feature.
- One zero count makes a whole score 0, because the score is a product; **Laplace smoothing** (G-1045) fixes it by adding 1 to every count.

11. Sources

Built from

- CampusX, "Naive Bayes Classifier | Part 6 | Intuition", YouTube, https://www.youtube.com/watch?v=
- StatQuest with Josh Starmer, "Naive Bayes, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=O2L2Uv9pdDA (likelihoods as histogram bars, scoring a short message, the zero count and the added counts, word order; Figures 1, 6 and 8 are redrawn from these ideas with our own numbers)

Other references

- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Section 4.4.4, pp. 153–155.
- **scikit-learn user guide:** Section 1.9, "Naive Bayes", https://scikit-learn.org/stable/modules/naive_b

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Naive Bayes classifier (Naive Bayes) (G-1297)	A classifier that applies Bayes' theorem with the assumption that inputs are independent within each class.
Naive assumption (G-1296)	The assumption that, within each class, the inputs are independent of each other (conditional independence given the class); it lets Naive Bayes multiply one probability per input instead of counting rare full combinations.
Score (G-1753)	In naive Bayes, the likelihood times the prior for one class; it is not the probability of the class, only proportional to it, which is enough to pick the class with the largest score.
Class prior (G-388)	The share of training rows in a class, such as 5 wins out of 8 matches; Naive Bayes uses it as the prior probability of that class.
Likelihood (G-1086)	How probable the observed data is under given parameter values; read as a function of the parameters with the data fixed.
Conditional probability (G-444)	The probability of an event given that another event has happened: $P(A \mid B)$.
Bayes' theorem (G-269)	The rule that reverses a conditional probability: from $P(B \mid A)$ it gives $P(A \mid B)$, so a belief about A can be updated after seeing B; $P(A \mid B) = P(B \mid A)P(A)/P(B)$.
Posterior (G-1536)	The probability of an event after the evidence is taken into account.
Evidence (G-718)	The overall chance of seeing what we observed, counted over every possible hypothesis, written $P(E)$. Bayes' theorem divides by it: posterior = likelihood × prior / evidence.
Prior (G-1565)	The probability of an event before any evidence is seen.
Joint probability (G-986)	The probability that two events happen together, $P(A \cap B)$.
Conditional independence (G-443)	Independence that holds once a third variable is known: given the class, knowing one feature tells nothing more about another. Naive Bayes assumes it so it can multiply one probability per feature.

Term	Meaning
Bag of words (G-250)	The simplest way to turn texts into feature vectors: list every unique word (the vocabulary), give each word one dimension, and count how often each word appears in the text.
Laplace smoothing (G-1045)	Adding a small count (usually 1) to every count so that no probability is 0.